

Comparing Clustering Algorithms

Rain Jocas

Introduction

In my Artificial Intelligence class at Bowdoin College, we explored a number of different machine learning tools, methods, and algorithms. One of these was classification, where the goal was to match data to the correct class or feature. For example, if a residence has 4 bedrooms and three bathrooms is it more likely to be a single family home or an apartment? The number of bedrooms and number of bathrooms are quantitative data that can be used to predict the qualitative category of residence type. In this scenario, the data has two dimensions: number of bedrooms and number of bathrooms. However more dimensions could be added, such as cost, distance from a main road, maximum number of residents, or more. For such a question, you could use machine learning classification algorithms such as knn, logistic regression, or perceptron. However all of these algorithms require you to know what the classes are already. What do you do then if the classes are unknown?

Clustering is a form of classification which groups items into clusters of other similar items. Unlike classic classification models, clustering does not require that you already know what the classes are (Cluster Analysis). Some forms of clustering require that you know the number of classes, while some do not. K-Means is a unique case where it can be implemented in such a way where you must already know the number of classes, or where you do not (K-Means Clustering). Because Clustering does not require knowledge of the specific classes, it opens up more possibilities for classification. Take for example our residential building scenario. What

would you do if you did not know all of the residence types? In that scenario you could use clustering algorithms to classify our data.

K-Means is a type of centroid based clustering algorithm (IBM). First it generates k centroids in random positions over the input data. For each point in the input data, it finds the centroid closest to that point. Each centroid is assigned to all the points closest to itself, and these points become a cluster. It recalculates the position of each centroid by averaging the positions of the points in the cluster it has been assigned to. Then it repeats everything but the first step either k times, or until centroid positions converge, depending on how the algorithm is implemented (K-Means Clustering).

DBSCAN is a type of density-based clustering algorithm (DBSCAN). Unlike K-Means, DBSCAN accounts for noise and does not classify every single point into a specific cluster (DBSCAN). First, it chooses a random point and finds all of the neighbors within epsilon distance of that point. If it does not have at least `minSamples` neighbors, then it is a noise point. If it has at least `minSamples` neighbors, it becomes a core point, and it and all of its neighbors are a cluster. It repeats this with all of the neighbors of that core point, and if any of those neighbors are core points, then their neighbors are added to the cluster, and so on until it cannot iterate further. It then repeats the entire algorithm until all points have been visited.

Hypothesis

For this problem, I wanted to compare the results of the k-Means and DBSCAN algorithms. I compared them based on measures of correctness and efficiency. I hypothesize that DBSCAN would be more efficient than k-means as it is non-iterative. I believe K-Means would be more accurate if K is equal or close to the true number of classes. However, in cases where k

is significantly smaller or larger than the number of classes, I hypothesize that DBSCAN would be more accurate. I had no hypothesis for how the data size would affect the results, but was interested in observing them nonetheless.

Methods

To test my hypothesis, I implemented a version of k-means where it does not iterate until convergence, but until the set number of iterations specified in the function call, which I set to K. This way, I could better test how the choice of K affected the results. If I had chosen to implement a version of K-Means based on convergence, then the runtime would be affected by the data set itself and not on the choice of K. As I was testing the algorithm and not the data, the latter was more useful. For my measure of distance, I used Euclidean distance. The DBSCAN algorithm I implemented was fairly standard, although I used recursion to find neighbors. Additionally, in order to account for any number of dimensions, my points were stored in a dictionary with their indexes as keys. I did, however, use Euclidean distance as my distance calculation. Should this be implemented to support more than two dimensions, that dimension calculation would need to be changed to account for more than the x,y axis.

At the end of each algorithm, I formatted them two different data types; A data frame with a column for clusters (and noise in the case of DBSCAN) and another column with lists of corresponding points, and a numpy array with a list of class assignments for each point (-1 corresponding to noise). This allowed me to print the result of my data in a format easy to read, as well as to use the numpy array to run metrics tests.

To test the correctness of my algorithms, I took the predicted classes output by each algorithm and compared them to the true classes. I used four different measures of correctness as

my metrics: Random-Index score, Mutual-Information score, homogeneity, and completeness. For these, I used the built in methods in sk-learn, rather than implementing them from scratch. Random-Index measures the similarity between actual and predicted classes (2.3 Clustering). It ignores permutations, so swapping class names does not affect the score (2.3 Clustering). It returns a value between 0 and 1, with the closer it is to 1 meaning the better the predictions are (2.3 Clustering). Mutual-Information measures the agreement between the true and predicted classes, also ignoring permutations (2.3 Clustering). It returns a value between 0 and 1, with 0 indicating class assignments are completely independent from each other, and 1 indicating true and predicted class assignments are in complete agreement (2.3 Clustering). The homogeneity test returns a value between 0 and 1, where 1 is fully homogeneous and 0 is not homogenous at all (2.3 Clustering). A homogenous result means each cluster only contains points from a single class, based on the true class assignments (2.3 Clustering). A complete result means all points of a given class are assigned to the same cluster, based on the true assignments (2.3 Clustering). The completeness test returns a value between 0 and 1, where 1 is fully complete and 0 is not complete at all (2.3 Clustering). I used the built in `make_blobs` data set to test these algorithms, changing the number of states, but keeping the number of samples and centers consistent.

Results

The results of this experiment surprised me significantly. On a data set of size 10, DBSCAN was significantly faster than k-means for all combinations of K, epsilon, and minSamples which I tested. Completeness was consistently high for both algorithms at size 10. Homogeneity was also relatively, but 0 for K-Means when K was equal to 0, and 0 for DBSCAN when epsilon was 10. For K-Means, the mutual information score increased consistently with k,

even when k was larger than the true number of classes. For DBSCAN, mutual information was 0 when epsilon was low, increased as epsilon increased, then decreased back to 0. For K-Means, random-index followed the same trend as mutual information. For DBSCAN, the random index increased until epsilon was 5 (the true number of classes), at which point it decreased with the increase of epsilon.

On a data set of size 100, DBSCAN still mostly outperformed K-Means based on runtime. However when minSamples was raised above 26, DBSCAN's runtime suddenly increased sharply to the point it took well over half an hour before I gave up on getting results from that test. For lower values of K , DBSCAN outperformed K-Means on random-index, mutual information, and completeness, while k-means outperformed DBSCAN on homogeneity. However as soon as K was raised to 10 and above, K-Means began to outperform DBSCAN on everything except completeness. Although slower, for a data set of size 100, K-Means outperformed DBSCAN in terms of measures of correctness, and once minSamples was raised to 26 and above, outperformed DBSCAN on runtime as well. On a data set of size 1000, trends remained similar to that of a data set of size 100. DBSCAN was significantly faster than k-means until minSamples was raised to 26 and above, at which point it was dramatically slower. K-Means measures of correctness raised until k reached 5 and above, at which point they became inconsistent in how they increased or decreased, although they stayed above 0.75 across the board. DBSCAN metrics, however, decreased the higher minSamples was.

Conclusion

DBSCAN is only more efficient than k-means in very specific scenarios. It requires either a small data set or particular values of minSamples to run both efficiently and correctly. For any

value of minSamples 26 or above, DBSCAN ran egregiously slowly compared to K-Means. This was likely due to the recursive nature of finding clusters, something which I was not aware of when forming my hypotheses. This recursive nature is probably why smaller values of minSamples tended to work better despite the choice of epsilon not affecting runtime. Additionally, the correctness scores of DBSCAN were variable and highly dependent on the choice of epsilon and minSamples. For K-Means, it was slower, but more reliable. A high value of K high metrics across the board, and a random index score of at least 0.75, regardless of the distance from the true number of classes. Larger values of k meant a slower algorithm, but predictable so. $K = 1000$ (~53 secs) took about 100 times longer than $k = 100$ (~0.53 secs), which itself took about 100 times longer than $K = 10$ (0.008628 secs) on a data set of size 1000. Overall, K-Means returned better results, and was more reliably speedy. It would be interesting to see how more intentional choices of epsilon and minSamples affected the results of this experiment.

Works Cited

“2.3 Clustering.” *Scikit Learn*, Scikit-learn Developers,

<https://scikit-learn.org/stable/modules/clustering.html>.

“Cluster Analysis.” *Wikipedia*, Wikimedia Foundation, 6 Dec. 2024,

en.wikipedia.org/wiki/Cluster_analysis.

“DBSCAN.” *Wikipedia*, Wikimedia Foundation, 19 Nov. 2024, en.wikipedia.org/wiki/DBSCAN.

Ibm. “What Is K-Means Clustering?” *IBM*, 27 Nov. 2024,

www.ibm.com/think/topics/k-means-clustering.

“K-Means Clustering.” *Wikipedia*, Wikimedia Foundation, 27 Nov. 2024,

en.wikipedia.org/wiki/K-means_clustering.